

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа № 2

За седьмой семестр

Тема: «Многослойный перцептрон для задачи «исключающее ИЛИ» с числовыми классами:
сравнение функций потерь MSE и Binary Cross-Entropy»

Выполнил:

Студент 4 курса

Группы ИИ-26

Петручик Д. А.

Проверил:

Никитин С.К.

Вариант 10

Цель: На основе многослойного персептрона архитектуры 2-2-1, реализованного в Лабораторной работе №1 для задачи XOR с числовыми классами c_0 и c_1 из диапазона $[-10; 10]$, реализовать альтернативную функцию потерь – бинарную кросс-энтропию (Binary Cross-Entropy, BCE) – и провести сравнительный анализ обучения сети с MSE (Лабораторная работа №1) и с BCE по скорости сходимости, качеству обучения и устойчивости к выбору начальных весов.

Вариант: тот же, что и в Лабораторной работе №1

Задачи лабораторной работы

1. Взять за основу код многослойного персептрона 2-2-1 из Лабораторной работы №1.
2. Поскольку BCE определена для выходов в диапазоне $(0; 1)$, а целевые значения в вашем варианте – произвольные числа c_0 и c_1 из диапазона $[-10; 10]$, реализовать нормализацию (масштабирование) целевых значений перед обучением и обратное преобразование выхода сети после обучения:
 - привести c_0 и c_1 к бинарным меткам 0 и 1;
 - обучать сеть на нормализованных целевых значениях с функцией потерь BCE;
 - при выводе результата преобразовывать выход сети $\hat{y} \in (0; 1)$ обратно в шкалу $[c_0; c_1]$.
3. Реализовать корректный градиент для связки BCE + сигмоида на выходном слое; для скрытого слоя градиент передаётся по стандартному правилу цепочки.
4. Обучить сеть на полной таблице истинности своего варианта (4 примера) в двух конфигурациях:
 - **Конфигурация А (базовая, из ЛР №1):** MLP 2-2-1, функция потерь MSE, целевые значения – исходные c_0/c_1 , выходной слой – сигмоида. Результаты можно взять из отчёта по ЛР №1 или, при необходимости, повторить обучение с фиксацией всех метрик.
 - **Конфигурация Б (новая):** MLP 2-2-1, функция потерь BCE, целевые значения – нормализованные (0/1), с последующим пересчётом выхода в шкалу $[c_0; c_1]$.

Критерий остановки для обеих конфигураций одинаков – суммарная ошибка $E_s \leq E_e$ (используйте то же значение E_e , что и в ЛР №1, либо согласуйте новое значение отдельно для MSE и для BCE, поскольку это разные по своей природе величины – обоснуйте выбор в отчёте).

5. Для каждой конфигурации зафиксировать:
 - количество эпох до достижения критерия остановки (либо факт недостижения за максимально допустимое число эпох);
 - итоговую суммарную ошибку;
 - ассигасу на всех 4 примерах таблицы истинности;
 - среднюю абсолютную ошибку в исходной шкале $[c_0; c_1]$.

6. Провести не менее 5 запусков обучения для каждой конфигурации с разной случайной инициализацией весов (менять seed) и сравнить устойчивость сходимости: в скольких из 5 запусков сеть достигла критерия останова, и какой разброс по числу эпох наблюдается.
7. Построить общий график зависимости суммарной ошибки от номера эпохи для конфигурации А (MSE) и конфигурации Б (VCE) на одних осях (для представительного запуска каждой конфигурации).
8. Реализовать блок визуализации результатов, включающий:
 - **График сходимости** – суммарная ошибка E_s от номера эпохи, конфигурации А и Б на одних осях, разными цветами/стилями линий, с обозначением порога E_e .
 - **Диаграмму разброса числа эпох по 5 запускам** (bar chart или scatter) для конфигурации А и конфигурации Б – наглядно показывающую устойчивость сходимости из п. 6 (например, столбики числа эпох для каждого из 5 seed, сгруппированные по конфигурации; запуски, не достигшие критерия останова, отметить отдельно, например штриховкой или отдельным цветом).
 - **Визуализацию разделяющей поверхности скрытого слоя** – для представительного обученного экземпляра каждой конфигурации построить на плоскости (А, В) карту выходного значения сети (heatmap или contour plot) с наложенными точками обучающей выборки (4 примера таблицы истинности), закрашенными по истинному классу. Диапазон осей А и В – $[-10; 10]$ (весь допустимый диапазон входных значений). Для конфигурации Б цветовую шкалу можно строить как по нормализованному $\hat{y} \in (0; 1)$, так и по пересчитанному значению в шкале $[c_0; c_1]$ – выбрать более наглядный вариант и обосновать выбор.
 - **Диаграмму сравнения точности восстановления шкалы** – столбчатую диаграмму средней абсолютной ошибки для конфигурации А и конфигурации Б, чтобы наглядно сопоставить точность регрессии к исходной шкале $[c_0; c_1]$.

Каждый график должен иметь подписанные оси, заголовок и легенду (где применимо). Разделяющую поверхность и графики точности рекомендуется разместить рядом (например, subplot 2×2 или отдельными рисунками) для удобства сравнения в отчёте.

9. Реализовать режим функционирования сети: пользователь вводит пару чисел (А, В) из диапазона $[-10; 10]$ (не обязательно совпадающих с c_0 и c_1). Сеть должна выводить:

- выход в нормализованной шкале $\hat{y} \in (0; 1)$ (с точностью до 4 знаков);
- выход, пересчитанный в исходную шкалу $[c_0; c_1]$ по формуле из п. 2;
- к какому классу (c_0 или c_1) ближе полученное значение.

Продемонстрировать работу режима функционирования на всех 4 примерах исходной таблицы истинности и не менее чем на 2–3 дополнительных парах (А, В), не входящих в обучающую выборку.

Вариант 10 ($c_0 = -3$, $c_1 = -8$)

А	В	$A \oplus B$
-3	-3	-3
-3	-8	-8
-8	-3	-8
-8	-8	-3

Код программы:

```
import argparse
from pathlib import Path

import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
from matplotlib.patches import Patch
import numpy as np

C0 = -3.0
C1 = -8.0
LEARNING_RATE = 0.1
MAX_EPOCHS = 20_000
SEEDS = (0, 1, 2, 3, 4)
LIMITS = {"A": 0.01, "B": 0.05}
X_REAL = np.array([[C0, C0], [C0, C1], [C1, C0], [C1, C1]])
Y_REAL = np.array([[C0], [C1], [C1], [C0]])
Y_BIN = (Y_REAL - C0) / (C1 - C0)
COLORS = {"A": "#2369a1", "B": "#d87519"}

def encode_inputs(values):
    return 2.0 * (np.asarray(values, dtype=float) - C0) / (C1 - C0) - 1.0

def decode_output(output):
    return C0 + (C1 - C0) * output

def nearest_class(values):
    values = np.asarray(values)
    return np.where(np.abs(values - C0) <= np.abs(values - C1), C0, C1)

def sigmoid(x):
    return np.exp(-np.logaddexp(0.0, -x))

class MLP:
    def __init__(self, seed):
        rng = np.random.default_rng(seed)
        self.W1 = rng.uniform(-0.5, 0.5, (2, 2))
        self.b1 = np.zeros((1, 2))
        self.W2 = rng.uniform(-0.5, 0.5, (2, 1))
        self.b2 = np.zeros((1, 1))

    def forward(self, x):
```

```

hidden = sigmoid(x @ self.W1 + self.b1)
logits = hidden @ self.W2 + self.b2
return hidden, logits, sigmoid(logits)

def predict(self, x):
    return self.forward(x)[2]

def update(self, x, y, mode, lr):
    hidden, _, output = self.forward(x)
    if mode == "A":
        delta = 2.0 * (output - y) * output * (1.0 - output)
    else:
        delta = output - y
    hidden_delta = (delta @ self.W2.T) * hidden * (1.0 - hidden)
    self.W2 -= lr * (hidden.T @ delta)
    self.b2 -= lr * delta
    self.W1 -= lr * (x.T @ hidden_delta)
    self.b1 -= lr * hidden_delta

def total_error(model, x, targets, mode):
    _, logits, output = model.forward(x)
    if mode == "A":
        return float(np.sum((output - targets) ** 2))
    return float(np.sum(np.logaddexp(0.0, logits) - targets * logits))

def assess(model, mode):
    output = model.predict(encode_inputs(X_REAL))
    values = output if mode == "A" else decode_output(output)
    return {
        "accuracy": float(np.mean(nearest_class(values) == Y_REAL) * 100.0),
        "mae": float(np.mean(np.abs(values - Y_REAL))),
        "values": values.ravel(),
    }

def train(seed, mode, lr=LEARNING_RATE, max_epochs=MAX_EPOCHS):
    model = MLP(seed)
    x = encode_inputs(X_REAL)
    targets = Y_REAL if mode == "A" else Y_BIN
    history = [total_error(model, x, targets, mode)]
    epoch = 0
    for epoch in range(1, max_epochs + 1):
        for i in range(len(x)):
            model.update(x[i:i + 1], targets[i:i + 1], mode, lr)
        history.append(total_error(model, x, targets, mode))
        if history[-1] <= LIMITS[mode]:

```



```

ax.set(yscale="log", xlabel="Завершённые эпохи (0 — до обучения)",
       ylabel="Суммарная ошибка Es (логарифмическая шкала)",
       title=f"Сходимость двух конфигураций, seed={seed}")
ax.grid(alpha=0.2)
ax.legend()
finish_plot(fig, directory, "01_convergence.png")

fig, ax = plt.subplots(figsize=(9, 5), layout="constrained")
positions = np.arange(len(SEEDS))
for mode, shift in (("A", -0.2), ("B", 0.2)):
    subset = [r for r in runs if r["mode"] == mode]
    bars = ax.bar(positions + shift, [r["epochs"] for r in subset],
                  width=0.4, color=COLORS[mode], label=f"Конфигурация {mode}")
    for bar, r in zip(bars, subset):
        if not r["converged"]:
            bar.set_hatch("////")
            bar.set_edgecolor("black")
ax.set(xticks=positions, xticklabels=[str(s) for s in SEEDS], xlabel="Seed",
       ylabel="Эпохи до порога или до лимита",
       title="Устойчивость сходимости в пяти запусках")
handles, labels = ax.get_legend_handles_labels()
ax.legend(handles + [Patch(facecolor="white", hatch="////", edgecolor="black")],
          labels + ["Порог не достигнут"])
finish_plot(fig, directory, "02_epochs.png")

axis = np.linspace(-10, 10, 250)
grid_a, grid_b = np.meshgrid(axis, axis)
grid = encode_inputs(np.column_stack([grid_a.ravel(), grid_b.ravel()]))
fig, axes = plt.subplots(1, 2, figsize=(12, 5), layout="constrained")
point_cmap = ListedColormap(["#ffe15b", "#992878"])
for ax, mode in zip(axes, ("A", "B")):
    output = picked[mode]["model"].predict(grid).reshape(grid_a.shape)
    values = output if mode == "A" else decode_output(output)
    limits = (0.0, 1.0) if mode == "A" else (C1, C0)
    surface = ax.pcolormesh(grid_a, grid_b, values, shading="auto",
                           cmap="viridis", vmin=limits[0], vmax=limits[1])
    fig.colorbar(surface, ax=ax, label="Выход сигмоиды" if mode == "A"
                 else "Выход после пересчёта:  $-3 - 5p$ ")
    boundary = (C0 + C1) / 2.0
    if values.min() < boundary < values.max():
        ax.contour(grid_a, grid_b, values, levels=[boundary],
                   colors="white", linewidths=1.5)
    ax.scatter(X_REAL[:, 0], X_REAL[:, 1], c=Y_BIN.ravel(),
               cmap=point_cmap, vmin=0, vmax=1, s=95, edgecolors="black")
    ax.set(xlim=(-10, 10), ylim=(-10, 10), xlabel="Вход A", ylabel="Вход B",
           title=f"Конфигурация {mode}, seed={seed}", aspect="equal")
    ax.legend(handles=[Patch(facecolor="#ffe15b", label="Истинный c0 = -3"),
                       Patch(facecolor="#992878", label="Истинный c1 = -8")],

```

```

        loc="upper right", fontsize=8)
fig.suptitle("Карта выхода сети; белая линия — граница классов  $\hat{y} = -5,5$ ")
finish_plot(fig, directory, "03_surfaces.png")

fig, ax = plt.subplots(figsize=(7, 5), layout="constrained")
means = [np.mean([r["mae"] for r in runs if r["mode"] == mode])
          for mode in ("A", "B")]
bars = ax.bar(["А: исходные цели + MSE", "Б: метки 0/1 + BCE"], means,
              color=[COLORS["A"], COLORS["B"]])
ax.bar_label(bars, fmt="%.5f", padding=4)
ax.set(ylabel="MAE в исходной шкале, среднее по пяти запускам",
       xlabel="Конфигурация", title="Точность восстановления числовых классов",
       ylim=(0, max(means) * 1.18))
finish_plot(fig, directory, "04_mae.png")

```

```

def predict_pair(model, a, b):
    p = model.predict(encode_inputs([[a, b]])).item()
    value = decode_output(p)
    label = nearest_class(value).item()
    return p, value, label

def show_examples(model):
    pairs = [tuple(row) for row in X_REAL] + [(-2, -7), (-7, -2), (-9, -9)]
    print("\nРежим функционирования конфигурации Б:")
    print("  A      B      p       $\hat{y}$   Класс")
    for a, b in pairs:
        p, value, label = predict_pair(model, a, b)
        print(f"{a:8.1f} {b:8.1f} {p:7.4f} {value:9.4f} {label:8.0f}")

```

```

def interactive_mode(model):
    print("\nВведите A B из [-10; 10]; q — выход.")
    while True:
        try:
            line = input("A B: ").strip()
            if line.lower() in {"q", "exit", "выход"}:
                break
        try:
            a, b = map(float, line.replace(",", " ").split())
        except ValueError:
            print("Нужны два числа через пробел, например: -3 -8.")
            continue
        if not (np.isfinite(a) and np.isfinite(b)
                and -10 <= a <= 10 and -10 <= b <= 10):
            print("Введите конечные числа в диапазоне [-10; 10].")
            continue

```

```

    p, value, label = predict_pair(model, a, b)
    class_name = "c0" if label == C0 else "c1"
    print(f"p = {p:.4f};  $\hat{y}$  = {value:.4f}; {class_name} = {label:g}")
except (EOFError, KeyboardInterrupt):
    break
print("Работа завершена.")

```

```

def main():
    parser = argparse.ArgumentParser(description="ЛП №2, вариант 10: MSE и BCE")
    parser.add_argument("--no-input", action="store_true")
    parser.add_argument("--no-show", action="store_true")
    parser.add_argument("--output-dir", type=Path, default=Path("lab2_results"))
    args = parser.parse_args()
    args.output_dir.mkdir(parents=True, exist_ok=True)
    print(f" c0={C0:g}, c1={C1:g}; lr={LEARNING_RATE}; "
          f"лимит={MAX_EPOCHS}; Ee(A)={LIMITS['A']}; Ee(B)={LIMITS['B']}")
    print("A: Es = сумма квадратов; B: Es = сумма BCE. Это разные шкалы потерь.")
    print("Для A Es > 146: сигмоида не может воспроизвести отрицательные цели.")
    runs = []
    for mode in ("A", "B"):
        for seed in SEEDS:
            result = train(seed, mode)
            runs.append(result)
            print(f"Завершено: конфигурация {mode}, seed={seed}, "
                  f"эпох={result['epochs']}, Es={result['loss']:.8f}", flush=True)
    print_results(runs)
    save_results(runs, args.output_dir)
    draw_plots(runs, args.output_dir, seed=0)
    representative = next(r for r in runs if r["mode"] == "B" and r["seed"] == 0)
    if not representative["converged"]:
        print("Внимание: представительный запуск Б не достиг порога.")
    show_examples(representative["model"])
    print(f"\nТаблица и четыре графика сохранены: {args.output_dir.resolve()}")
    if args.no_show:
        plt.close("all")
    else:
        plt.show()
    if not args.no_input:
        interactive_mode(representative["model"])

if __name__ == "__main__":
    main()

```

Результат работы программы:

○ $c_0=-3$, $c_1=-8$; $lr=0.1$; лимит=20000; $E_e(A)=0.01$; $E_e(B)=0.05$

А: E_s = сумма квадратов; Б: E_s = сумма ВСЕ. Это разные шкалы потерь.

Для А $E_s > 146$: сигмоида не может воспроизвести отрицательные цели.

Завершено: конфигурация А, seed=0, эпох=20000, $E_s=146.00017911$

Завершено: конфигурация А, seed=1, эпох=20000, $E_s=146.00017539$

Завершено: конфигурация А, seed=2, эпох=20000, $E_s=146.00017710$

Завершено: конфигурация А, seed=3, эпох=20000, $E_s=146.00017508$

Завершено: конфигурация А, seed=4, эпох=20000, $E_s=146.00017644$

Завершено: конфигурация В, seed=0, эпох=2934, $E_s=0.04999270$

Завершено: конфигурация В, seed=1, эпох=2722, $E_s=0.04998190$

Завершено: конфигурация В, seed=2, эпох=3277, $E_s=0.04999061$

Завершено: конфигурация В, seed=3, эпох=20000, $E_s=1.91249171$

Завершено: конфигурация В, seed=4, эпох=3049, $E_s=0.04998572$

Конф.	Seed	Эпохи	E_s	Accuracy	MAE	Порог
А	0	20000	146.00017911	50.00%	5.50000410	не достигнут
А	1	20000	146.00017539	50.00%	5.50000404	не достигнут
А	2	20000	146.00017710	50.00%	5.50000404	не достигнут
А	3	20000	146.00017508	50.00%	5.50000398	не достигнут
А	4	20000	146.00017644	50.00%	5.50000408	не достигнут
В	0	2934	0.04999270	100.00%	0.06208535	достигнут
В	1	2722	0.04998190	100.00%	0.06208440	достигнут
В	2	3277	0.04999061	100.00%	0.06208297	достигнут
В	3	20000	1.91249171	75.00%	1.67830139	не достигнут
В	4	3049	0.04998572	100.00%	0.06208913	достигнут

А: сошлось 0/5 запусков.

Число эпох до сходимости не определено: порог не достигнут.

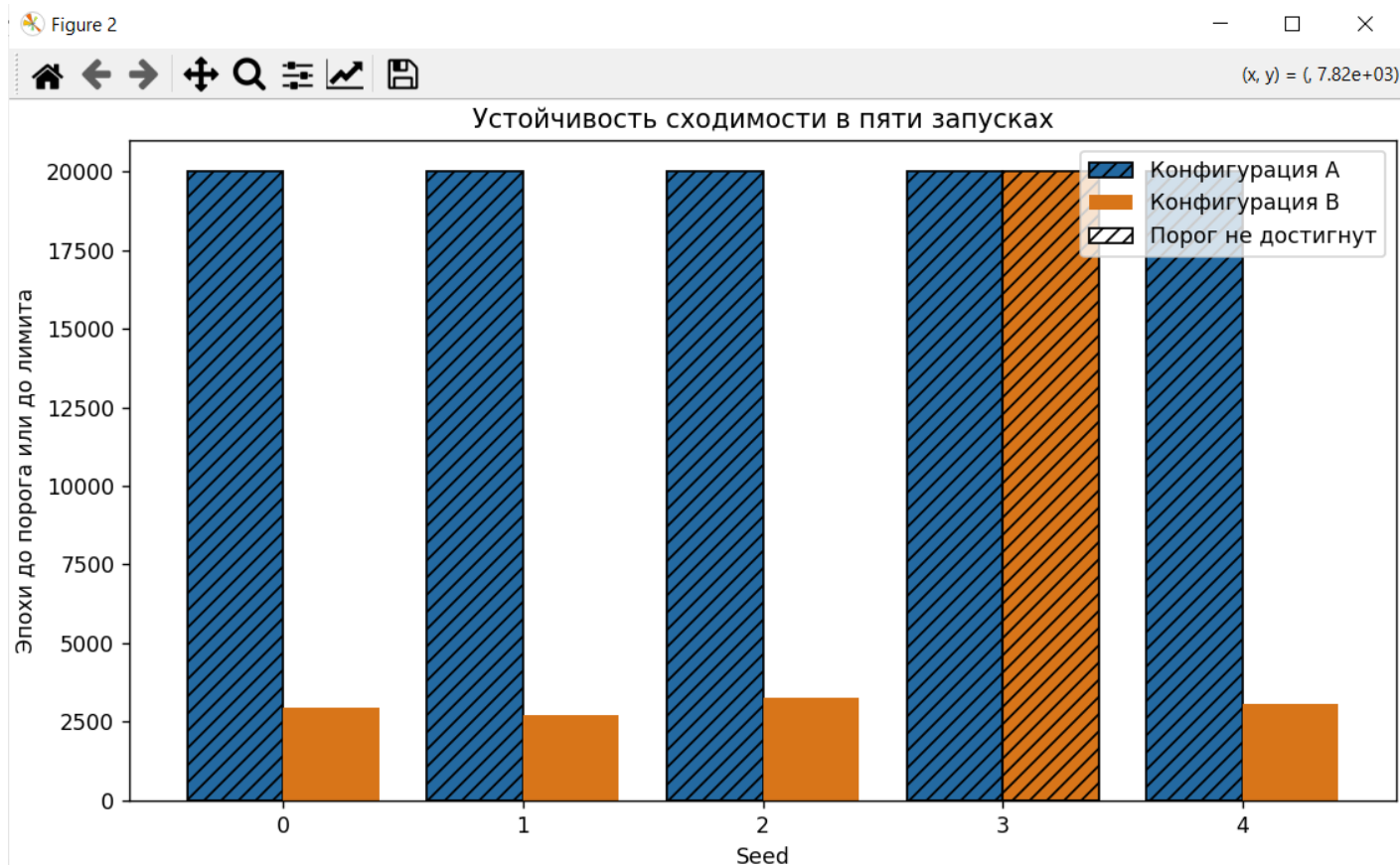
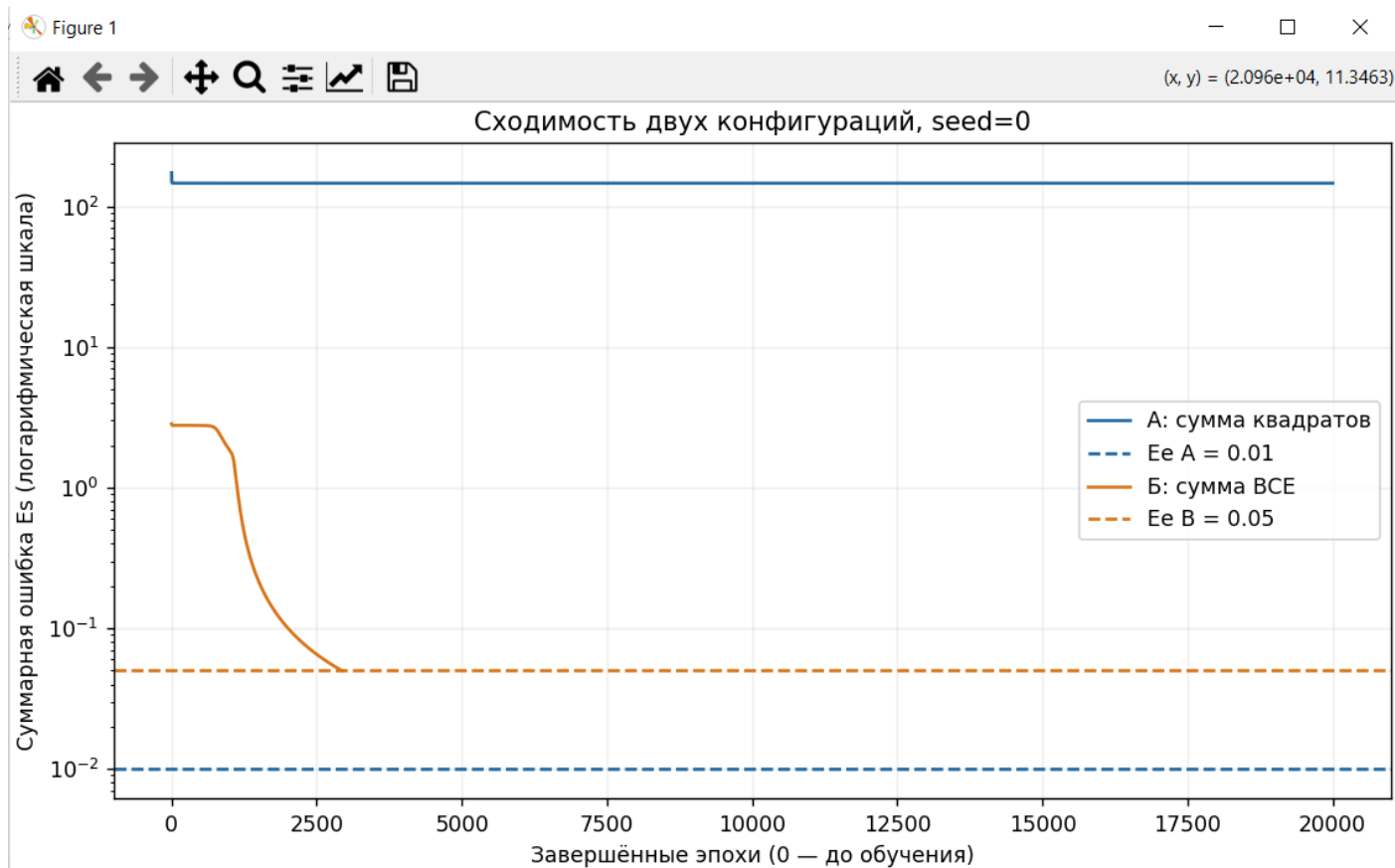
В: сошлось 4/5 запусков.

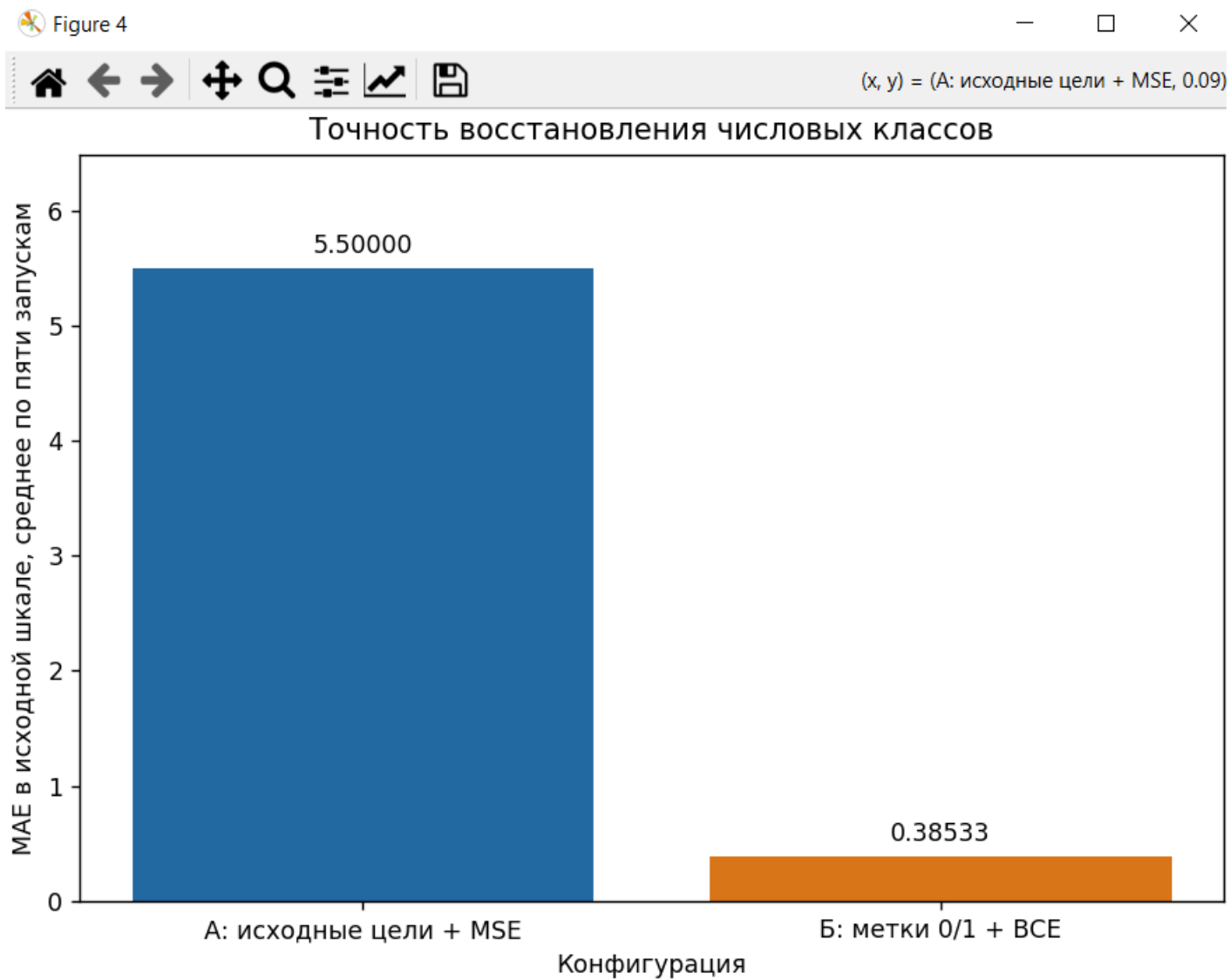
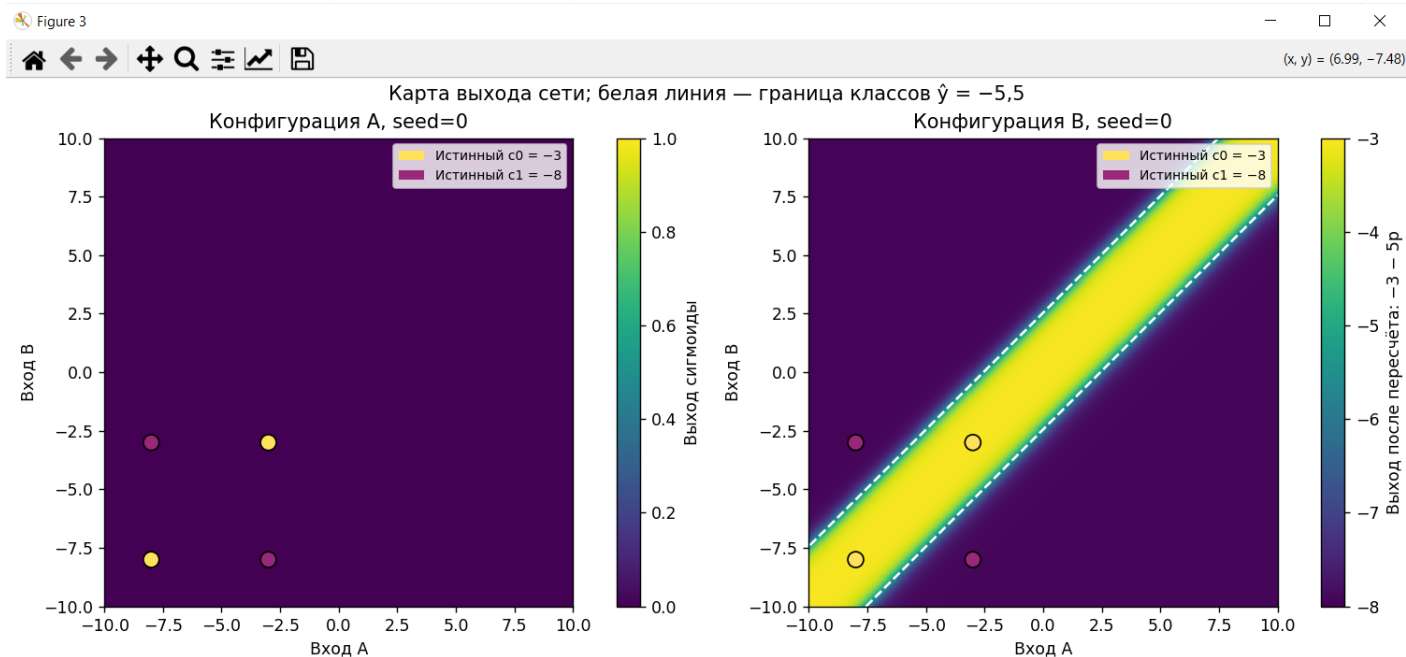
Эпохи успешных запусков: min=2722, max=3277, mean=2995.5, std=200.4, размах=555.

Режим функционирования конфигурации Б:

А	В	р	$\hat{y}$	Класс
-3.0	-3.0	0.0111	-3.0553	-3
-3.0	-8.0	0.9832	-7.9158	-8
-8.0	-3.0	0.9893	-7.9464	-8
-8.0	-8.0	0.0110	-3.0552	-3
-2.0	-7.0	0.9832	-7.9158	-8
-7.0	-2.0	0.9893	-7.9464	-8
-9.0	-9.0	0.0110	-3.0551	-3

Графики:





Вывод: В ходе лабораторной работы для варианта 10 с классами ($c_0 = -3$) и ($c_1 = -8$) были исследованы две конфигурации многослойного персептрона 2–2–1: обучение с MSE на исходных целевых значениях и обучение с BCE на нормализованных метках. Сравнение показало, что согласование целевых значений с диапазоном выходной функции активации необходимо для успешного обучения.

Вопросы:

1. Скорость и устойчивость сходимости MSE и BCE.

Конфигурация Б с BCE достигла критерия остановки в четырёх из пяти запусков за 2722–3277 эпох. Конфигурация А с MSE не достигла порога ни в одном запуске за 20 000 эпох. Для BCE с сигмоидой выходной градиент равен $(p-t)$, без дополнительного множителя $(p(1-p))$, который может замедлять обучение при квадратичной потере. Результат согласуется с преимуществами BCE для классификации. Однако в данном эксперименте одновременно изменялись функция потерь и кодирование целей, поэтому приписывать улучшение только BCE нельзя.

2. Влияние нормализации целевых значений.

Преобразование классов -3 и -8 в метки 0 и 1 согласовало цели с диапазоном сигмоиды. При прямом обучении на отрицательных значениях сигмоида, выход которой находится в интервале $((0;1))$, принципиально не может воспроизвести цели. После нормализации достижение заданного порога стало возможным, и четыре запуска из пяти завершились успешно. Следует учитывать, что в использованном коде ЛР №1 цели уже кодировались в 0/1 с последующим обратным пересчётом; прямое обучение на $-3/-8$ выполнялось именно в конфигурации А ЛР №2.

3. Точность восстановления исходной шкалы.

Для представительного запуска с $\text{seed} = 0$ конфигурация Б обеспечила $\text{MAE} \approx 0,062085$ и $\text{ассигасу} = 100\%$, тогда как конфигурация А — $\text{MAE} \approx 5,500004$ и $\text{ассигасу} = 50\%$. Средняя MAE по всем пяти запускам составила 0,385329 для Б и 5,500004 для А. Обратное преобразование ($y_{\{\text{real}\}} = -3-5p$) позволило восстановить исходную шкалу. Правильная классификация не означает нулевую числовую ошибку, однако общий компромисс между классификацией и регрессией по этому эксперименту установить нельзя: выход конфигурации А изначально несовместим с отрицательными целями.

4. Устойчивость к случайной инициализации.

Конфигурация А не сошла во всех пяти запусках по причине ограничения диапазона выхода. Конфигурация Б сошла при $\text{seed} = 0, 1, 2$ и 4. При $\text{seed} = 3$ после 20 000 эпох ассигасу составила 75%, а суммарная BCE — около 1,912492. Среди успешных запусков среднее число эпох равно 2995,5, размах — 555 эпох. Следовательно, обучение Б зависит от инициализации. Неудачный запуск может свидетельствовать о плато или неудачной области параметров, но сам по себе не доказывает попадание в строгий локальный минимум.

5. Линейная неразделимость XOR.

Замена логических значений числовыми классами -3 и -8 не изменила линейную неразделимость задачи. Точки одинакового класса расположены в противоположных вершинах квадрата, поэтому разделить классы одной прямой невозможно. Вывод ЛР №1 сохраняется независимо от конкретных значений, выбранных для кодирования двух классов.

6. Необходимость многослойной архитектуры.

Выбор функции потерь влияет на процесс обучения, но не заменяет скрытый слой. Один

сигмоидный нейрон формирует линейную границу классификации и не способен правильно разделить все четыре примера XOR ни при MSE, ни при BCE. Скрытый слой создаёт необходимое нелинейное преобразование входов; в данной работе для решения задачи достаточно архитектуры 2–2–1.

7. Интерпретация разделяющей поверхности.

У конфигурации А выход почти постоянен, и все точки относятся к ближайшему классу –3. У конфигурации Б для $\text{seed} = 0$ сформировалась диагональная полоса класса –3, содержащая точки с одинаковыми входами, и области класса –8 вне этой полосы, содержащие точки с различающимися входами. Полоса приближённо соответствует пересечению двух полуплоскостей, сформированных скрытыми нейронами. На четырёх обучающих точках такая поверхность корректно реализует XOR. Границы не обязаны образовывать четыре квадранта: таблица из четырёх примеров не задаёт единственную форму поверхности на всей плоскости.